AWS-bastion-host-secure-access

AWS Cloud Security / Networking Project

Prepared by:

Shubham Bhatti

1. Why I Built This

Most of the AWS tutorials I'd worked through before this project had one thing in common: every instance sat in a public subnet with a public IP, reachable straight from the internet. That's fine for a five-minute demo, but it's not how anyone would actually run a server in production, and it bothered me that I didn't have hands-on experience with the alternative.

So for this project I set out to build a small but realistic version of a pattern that's used everywhere in real AWS environments: a **bastion host**. The idea is simple — instead of giving every server a public IP, you put one "jump box" in a public subnet, lock it down, and route all administrative access through it. Everything else stays in a private subnet with no direct path in from the internet.

I built this using my own AWS account, working entirely through the console rather than automation, so that I could actually see and understand each piece before I try to script it later.

2. Aim

Design and deploy a bastion host architecture in AWS that lets an administrator reach an EC2 instance sitting in a private subnet, without giving that private instance a public IP of its own.

3. What I Set Out to Do

- Build a custom VPC from scratch rather than using the default one.
- Split it into a public subnet and a private subnet.
- Set up route tables so the two subnets behave differently — one with a path to the internet, one without.
- Put the bastion host in the public subnet.
- Put a second EC2 instance in the private subnet, with no public IP assigned.
- Actually verify that the setup works, not just that it looks right in the console.

4. Tools and AWS Services

Service / Technology	What it did in this project
AWS VPC	The isolated network everything else lives in
Subnets	Split public-facing resources from internal ones
Internet Gateway	Gives the public subnet a path to the internet
Route Tables	Decide where traffic from each subnet is allowed to go

Service / Technology	What it did in this project
EC2	Runs the bastion host and the private server
Security Groups	Firewall rules controlling inbound/outbound traffic
RDP	How I connected to the Windows bastion host
Windows Server	OS on both EC2 instances

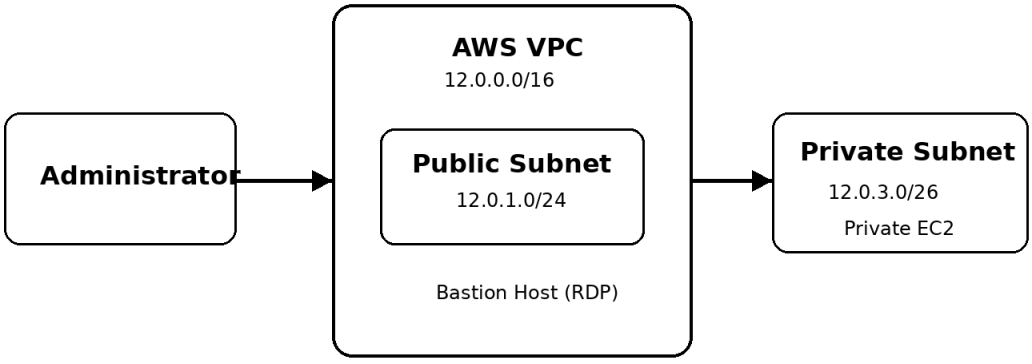
5. Network Layout

Component	Value
VPC	12.0.0.0/16
Public Subnet	12.0.1.0/24
Private Subnet	12.0.3.0/26
Availability Zone	us-east-1a
Public Route Table	demo-public-RT
Private Route Table	demo-private-RT
Bastion Host OS	Windows Server
EC2 Instance Type	t3.micro
Admin Protocol	RDP

6. Architecture

The public subnet holds the bastion host and routes out to the internet through the Internet Gateway. The private subnet holds the target EC2 instance and uses its own route table, one with no route to the Internet Gateway at all — so there's no way in except through the bastion.

Administrator → Bastion Host → Private EC2



7. Step-by-Step Build

Step 1 — Create the VPC

I started with a custom VPC, `demo-vpc`, using the CIDR block 12.0.0.0/16. Everything else in this project sits inside it.

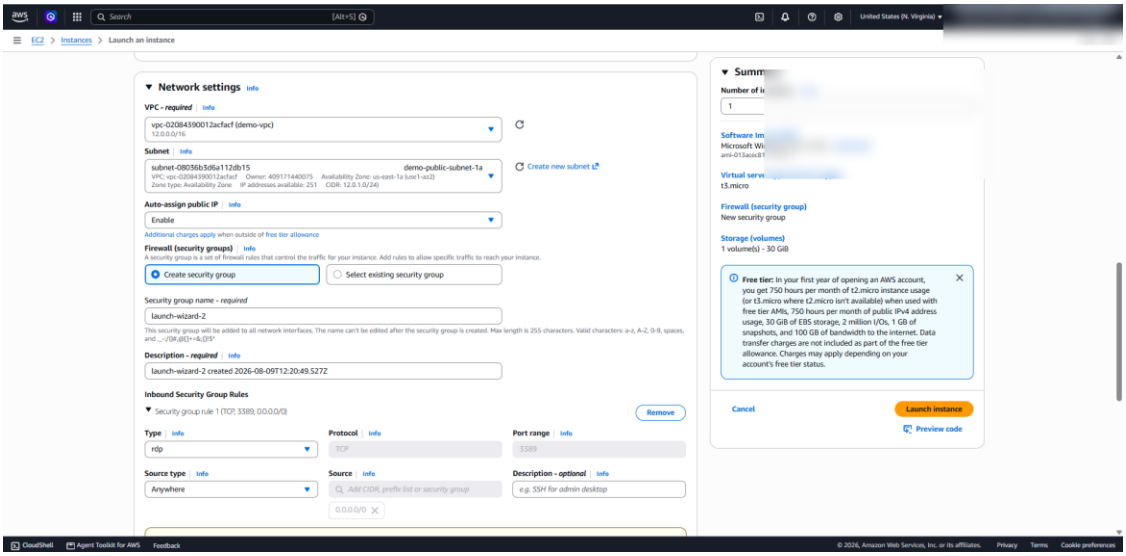


Figure 1 — VPC configuration.

Step 2 — Public Subnet

Created a public subnet at 12.0.1.0/24. This is where the bastion host lives, since it's the one resource that actually needs to be reachable from outside the VPC.

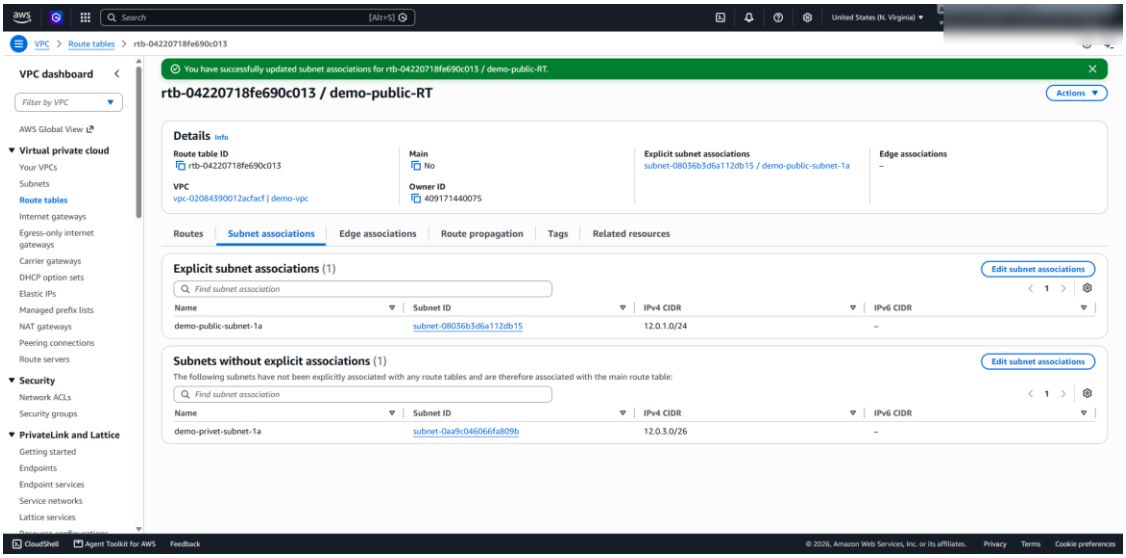


Figure 2— public subnet configuration.

Step 3 — Private Subnet

A second subnet, 12.0.3.0/26, for the private EC2 instance. I made sure this one doesn't get a public IPv4 address at all — the whole point of the exercise falls apart if it does.

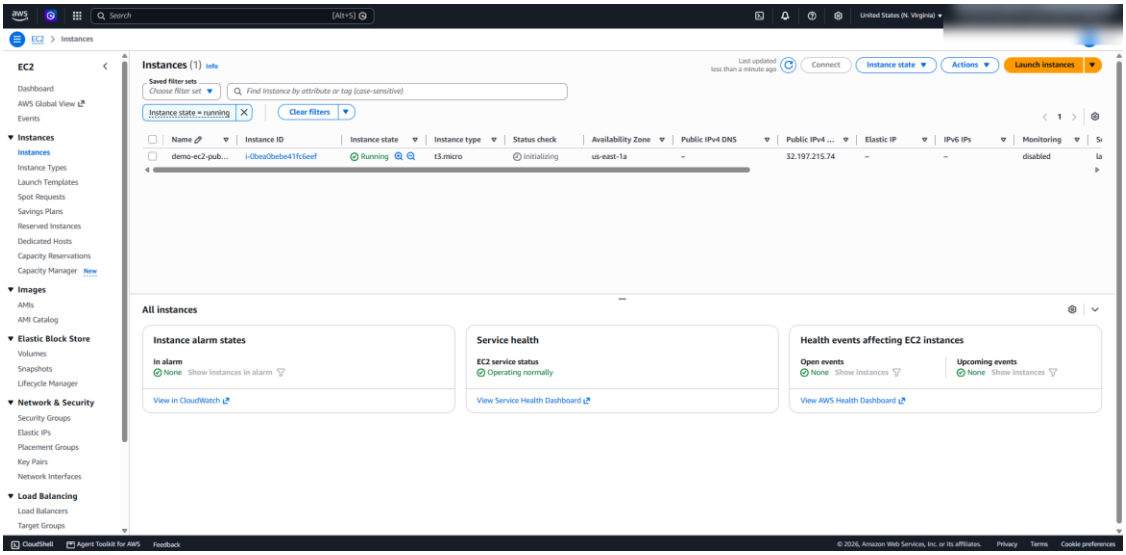


Figure 3 — private subnet / private EC2 configuration.

Step 4 — Public Route Table

Associated `demo-public-RT` with the public subnet and added a default route (0.0.0.0/0) pointing at the Internet Gateway. That's what actually gives the bastion its path out to (and in from) the internet.

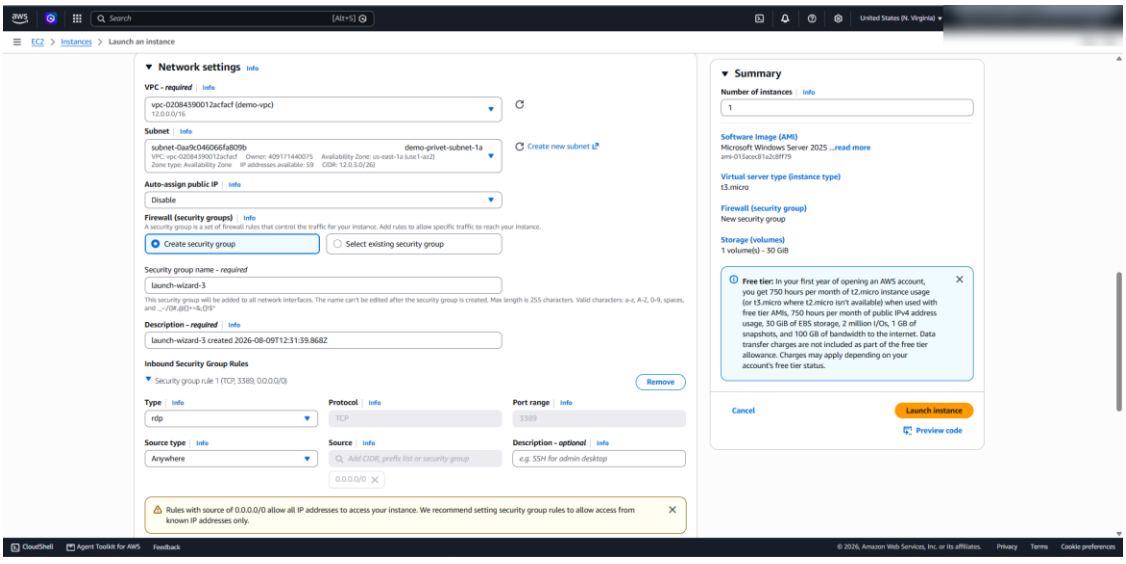


Figure 4 — public route table with the Internet Gateway route.

Step 5 — Private Route Table

`demo-private-RT` is associated with the private subnet, and deliberately has no default route to the Internet Gateway. This is really the core of the whole design — without this missing route, "private" subnet would just be a label.

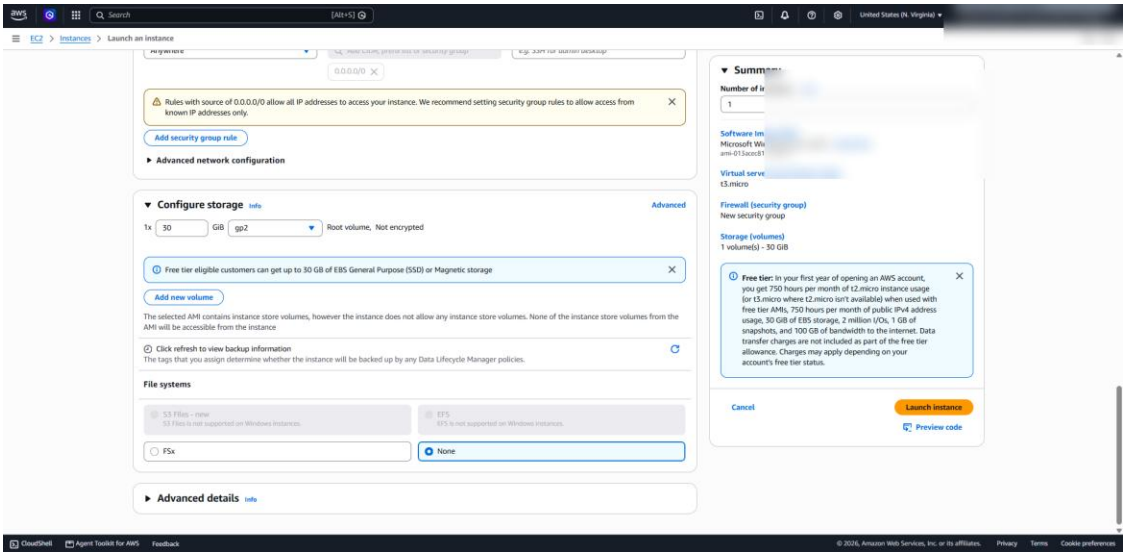


Figure 5 — private route table and subnet association.

Step 6 — Launch the Bastion Host

Launched a Windows Server EC2 instance (t3.micro) into the public subnet with a public IPv4 address, so I'd actually be able to RDP into it from my machine.

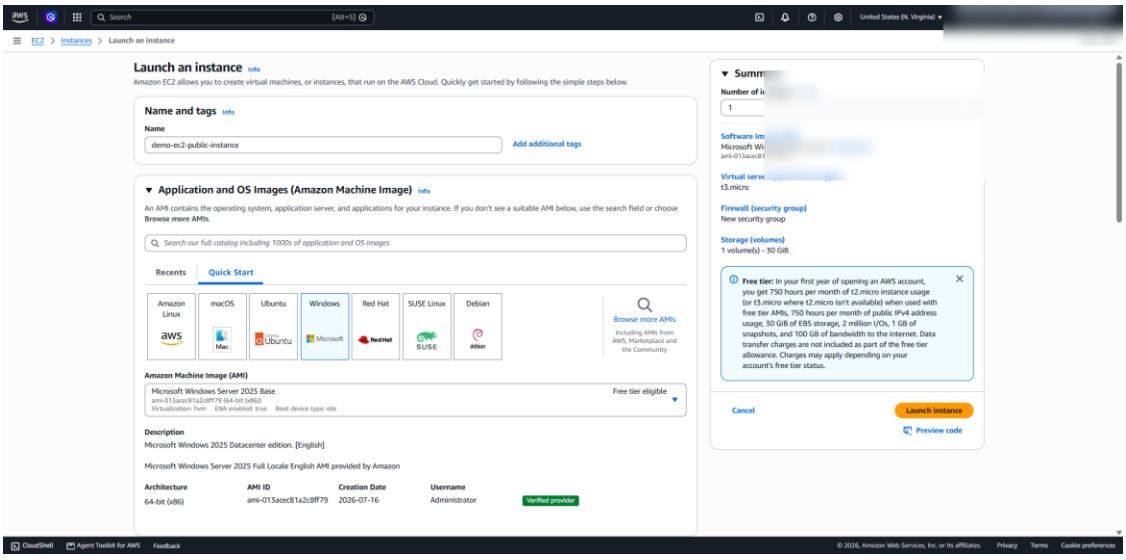


Figure 6 — bastion host EC2 configuration.

Step 7 — Bastion Security Group

Opened RDP (TCP 3389) on the bastion's security group. I'll be upfront about this one: for testing, I left the source as 0.0.0.0/0, which means literally any IP could attempt a connection. It worked, but it's exactly the kind of thing I'd never leave in place for a real deployment — more on that in the Limitations section below.

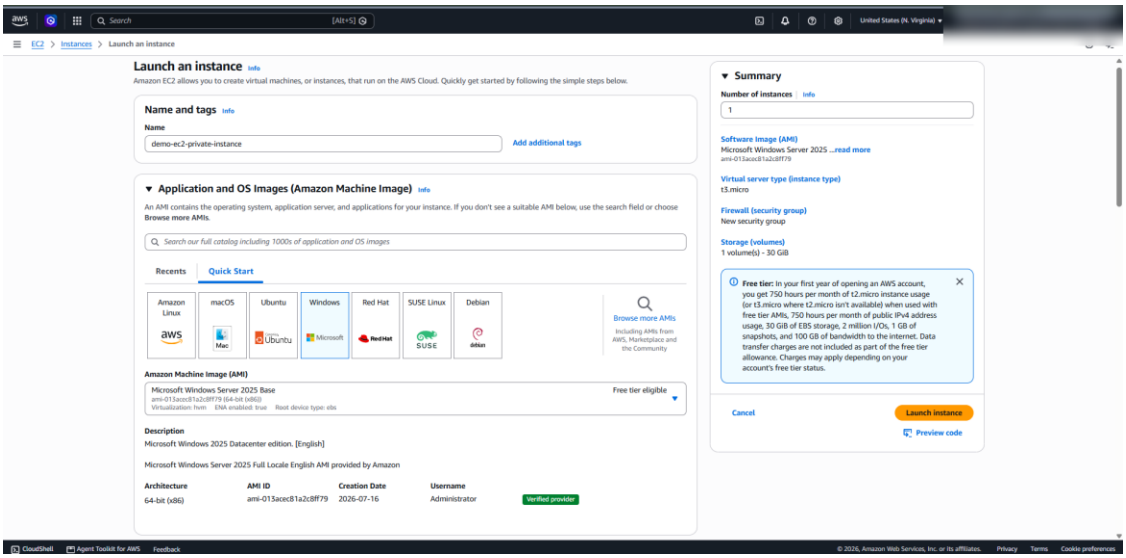


Figure 7 — bastion host security group configuration.

Step 8 — Launch the Private Instance

Launched a second Windows Server EC2 instance into the private subnet, this time with auto-assign public IP switched off, so it has no direct route in from the internet.

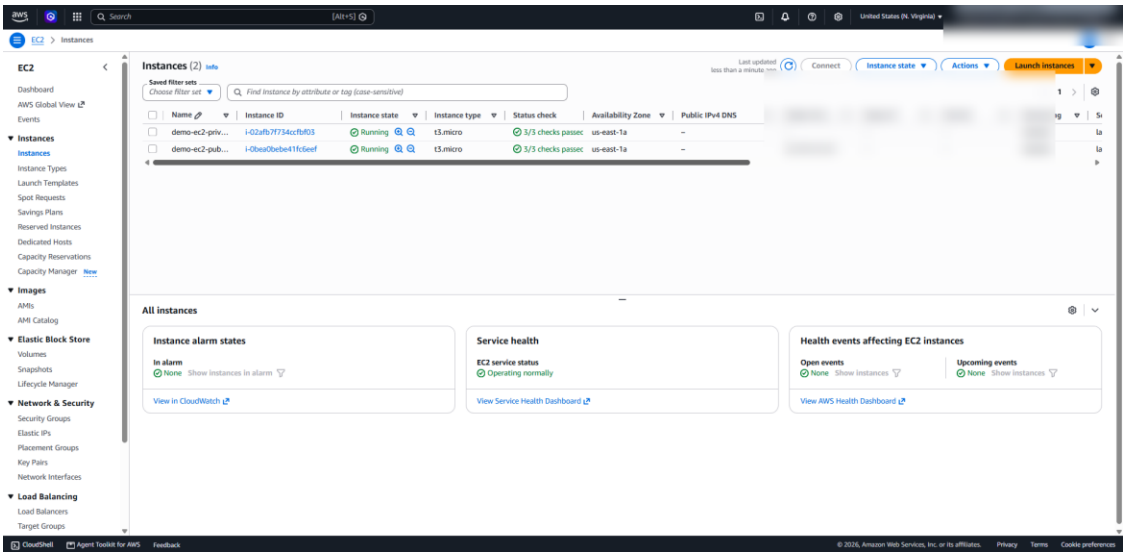


Figure 8 — private EC2 network configuration.

Step 9 — Verify the Bastion Works

Connected to the bastion over RDP using its public address to confirm the front door actually opens.

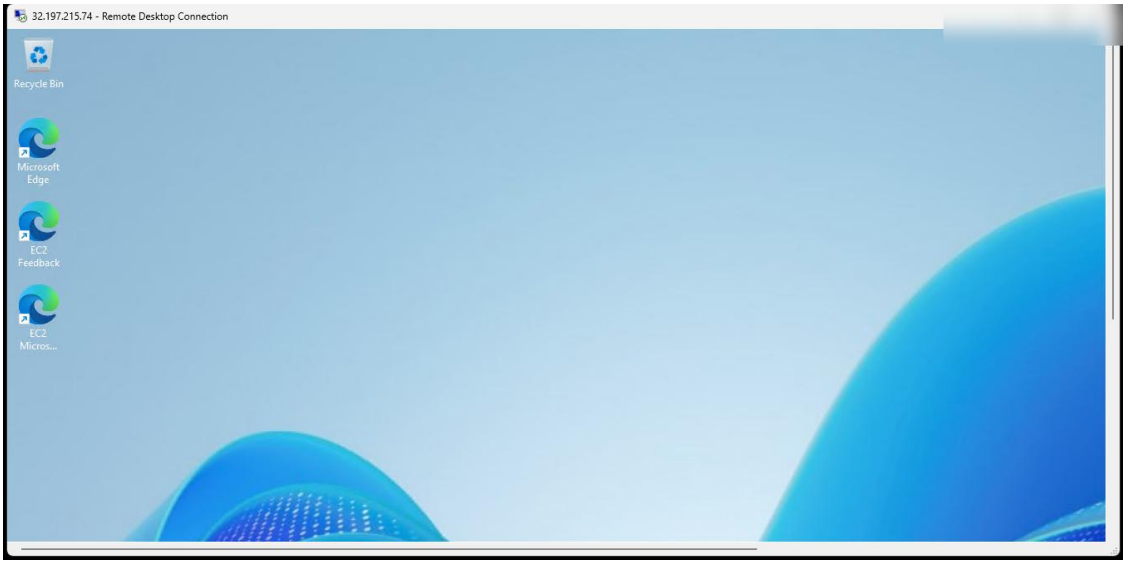


Figure 9 — successful RDP connection to the bastion.

8. Result

By the end of this, I had a working VPC split into public and private subnets, a bastion host sitting in the public subnet with a public IP, and a second EC2 instance in the private subnet with no public IP at all. The public route table pushes traffic out through the Internet Gateway; the private route table doesn't. I confirmed RDP access to the bastion actually works, which was the real test of whether any of this held together.

9. What Makes This More Secure

The core idea here isn't complicated — it's just consistent separation of what's public and what isn't:

- Public and private resources live in genuinely separate subnets, not just separate names.
- The private EC2 instance has no public IP to attack in the first place.
- Each subnet has its own route table, so the routing itself enforces the boundary.
- The Internet Gateway only touches the public routing path.
- The bastion is the single, controlled entry point for administrative access.
- Security groups filter traffic on top of the subnet-level isolation.
- Access to both instances is key-pair based rather than password-based.

10. Where This Falls Short (and Why I'm Flagging It)

I don't want to gloss over this: during testing, the bastion's RDP rule allowed connections from 0.0.0.0/0 — any IP on the internet could attempt to connect. That was a conscious shortcut to get the lab working quickly, not something I'd defend as good practice, and I'm calling it out here rather than cropping it out of the screenshots.

In an actual production setup, that RDP source should be locked down to a specific admin IP, a VPN CIDR range, or replaced entirely with something like AWS Systems Manager Session Manager, which removes the need for an open inbound port at all.

11. What I Took Away From This

This was a small project, but it gave me a much clearer picture of how public and private resources actually get separated in AWS — not conceptually, but in terms of the concrete pieces: subnets, route tables, security groups, and where public IPs do or don't get assigned.

The biggest lesson, honestly, was more about mindset than about AWS specifically: something can be fully functional and still not be secure. The 0.0.0.0/0 RDP rule worked perfectly for testing, and that's exactly why it's dangerous to leave in place — it "works," so it's tempting to just move on. Building this the manual way, one console screen at a time, made it much easier to see that gap than reading about it would have.

13. Notes on Sharing This Publicly

The screenshots in this report have had account-level details and specific addresses redacted before publishing. A few ground rules I followed, and would follow again:

- Never upload AWS credentials, passwords, private keys, `.pem` files, or anything else that counts as a secret.
- Keep the architecture diagram accurate to the real subnet CIDRs, even after redaction.
- Don't describe a lab setup as production-ready — it isn't, and this report says so directly.
- On LinkedIn, link back to the GitHub repo instead of re-uploading every screenshot there.
- Call out the temporary 0.0.0.0/0 RDP rule as a known limitation, with the fix, rather than leaving it unmentioned.